PROJETO · ENGENHARIA DE SOFTWARE

Sistema de Depósito de Materiais de Construção

Requisitos, MVP e código em Python + Tkinter



João Victor Oliveira Medeiros · Linguagem de Programação · 12/11/2026

CONTEXTO

O problema do depósito e o objetivo

DEMANDA 1

Difícil controlar o estoque físico de materiais volumosos e fracionados, como cimento, areia, tubos e tijolos.

DEMANDA 2

Perda de tempo no balcão para conferir preço e disponibilidade de cada item.

DEMANDA 3

Risco de faltar mercadoria sem ninguém perceber (tratado na próxima versão).

OBJETIVO

Aplicação desktop leve que cadastra materiais, registra entradas e saídas, permite consultar o estoque e mantém tudo salvo em arquivo.

REQUISITOS

Requisitos funcionais: o que o sistema faz

ID	Requisito funcional	Onde está no código
RF01	Cadastrar material (ID, nome, categoria, quantidade, preço)	adicionar_produto()
RF02	Listar todos os materiais em uma tabela	atualizar_tabela()
RF03	Registrar entrada e saída (baixa) de material	controle_estoque()
RF04	Impedir saída que deixe o estoque negativo	controle_estoque()
RF05	Buscar por nome ou ID e filtrar por categoria	filtrar_tabela()
RF06	Validar o que o usuário digita (vazio, preço, quantidade)	realizar_cadastro()
RF07	Salvar e carregar os dados em arquivo	salvar_dados(), ler_arquivo()

Fonte: Valente, Engenharia de Software Moderna, cap. 3 (Requisitos). Requisito funcional = funcionalidade do sistema.

REQUISITOS

Requisitos não funcionais: como o sistema se comporta

Usabilidade

Janela única com formulários, botões e tabela, para uso no balcão sem treinamento formal.

Meta: funcionário novo registra uma saída em até 1 minuto.

Portabilidade

Roda em qualquer computador com Python 3, sem servidor nem banco de dados.

Meta: nenhuma dependência externa (só tkinter e os).

Confiabilidade

Dados gravados em arquivo UTF-8 a cada cadastro ou movimentação.

Meta: nenhuma operação confirmada se perde ao fechar o programa.

Desempenho

A tabela mostra a alteração logo depois de cada operação.

Meta: atualização em até 1 segundo com 1.000 materiais.

Fonte: Valente, Engenharia de Software Moderna, cap. 3 (Requisitos). Requisitos não funcionais devem ser quantificados.

REQUISITOS

Histórias de usuário

HU1 · BALCONISTA

Como balconista, eu gostaria de buscar um material por nome ou ID e filtrar por categoria, para dizer ao cliente se há pronta-entrega e o preço.

Confirmação: digitar “cimento” lista só os materiais cujo nome contém o termo.

HU2 · GERENTE DO DEPÓSITO

Como gerente do depósito, eu gostaria de cadastrar novos materiais com ID, nome, categoria, quantidade e preço, para que passem a ser controlados.

Confirmação: preço zero ou negativo é recusado; o material aparece na tabela.

HU3 · GERENTE DO DEPÓSITO

Como gerente do depósito, eu gostaria de registrar a entrada de mercadorias, para manter a quantidade disponível sempre atualizada.

Confirmação: entrada de 20 unidades soma 20 ao saldo e salva no arquivo.

HU4 · BALCONISTA

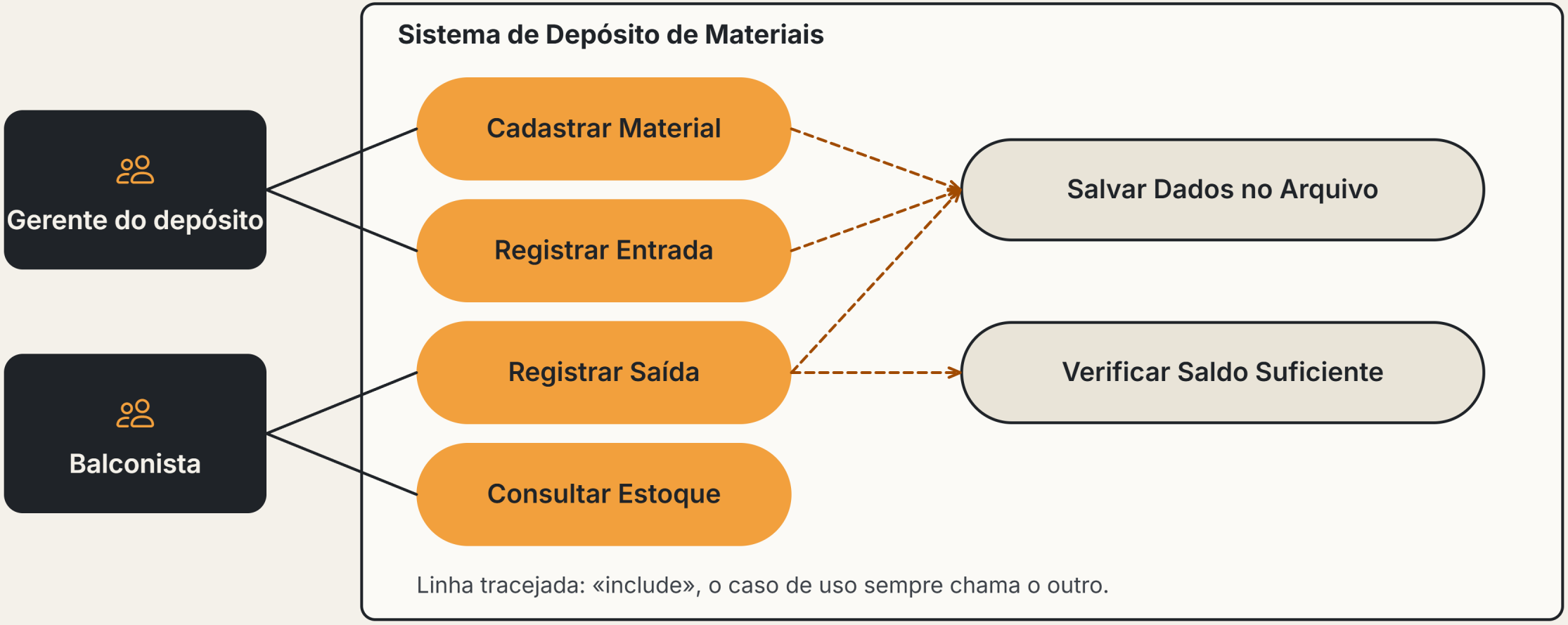
Como balconista, eu gostaria de dar baixa em uma venda, para que o estoque reflita o que saiu sem ficar negativo.

Confirmação: saída maior que o saldo é recusada e o saldo não muda.

Próximas histórias: HU5, estoquista avisado de estoque baixo; HU6, proprietário vê o valor total em estoque.

Fonte: Valente, Engenharia de Software Moderna, cap. 3 (Requisitos). História = cartão + conversas + confirmação.

Diagrama de casos de uso



Fonte: Valente, Engenharia de Software Moderna, cap. 3 (Requisitos). Atores, casos de uso e relações dentro da fronteira do sistema (UML).

CASO DE USO

Registrar Saída de Material

Ator: balconista. Pré-condição: o material já está cadastrado.

Fluxo normal

1. O balconista informa o ID do material.
2. O balconista informa a quantidade a retirar.
3. O balconista solicita a baixa.
4. O sistema confere se o saldo cobre a quantidade.
5. O sistema subtrai a quantidade e salva os dados.
6. O sistema atualiza a tabela e confirma a operação.

Extensões

1a, 2a. ID ou quantidade em branco: o sistema pede o preenchimento.

2b. Quantidade não é um inteiro maior que 0: o sistema recusa e avisa.

4a. ID inexistente ou saldo insuficiente: o sistema recusa, avisa a falha e mantém o saldo.

Pós-condição: saldo reduzido e gravado no arquivo.

Fonte: Valente, Engenharia de Software Moderna, cap. 3 (Requisitos). Caso de uso = ator + fluxo normal + extensões.

MVP

Produto mínimo viável

Hipótese: um depósito pequeno consegue controlar entradas, saídas e consultas em um sistema desktop simples, sem caderno nem planilha.

Dentro do MVP

- Cadastro de materiais
- Entrada e saída com saldo protegido
- Busca por nome ou ID e filtro por categoria
- Tabela do estoque
- Dados salvos em arquivo

Próxima versão

- Alerta de estoque mínimo
- Resumo do valor total em estoque
- Histórico de movimentações
- Bloqueio de ID duplicado

Construir, medir, aprender

- Construir: este sistema
- Medir: tempo para registrar uma venda e divergência entre prateleira e sistema
- Aprender: evoluir, ajustar ou pivotar

Fonte: Valente, Engenharia de Software Moderna, cap. 3 (Requisitos). MVP = menor sistema que testa uma hipótese de negócio.

ARQUITETURA

Três arquivos, cada um com sua função

interface.py

Tela em Tkinter: formulários, botões, tabela e mensagens ao usuário.

sistema.py

Regras do estoque: cadastro, movimentação, busca e validações.

dados_estoque.txt

Arquivo com um material por linha, campos separados por ponto e vírgula.

EXEMPLO DE LINHA NO ARQUIVO

```
3;Tinta Azul Marinho;Pintura;20;56.7
```

A tela chama as funções de sistema.py para alterar o estoque, então as regras ficam em um lugar só.

CÓDIGO 1 · sistema.py · RF01

Modelo de dados e cadastro

```
CATEGORIAS = ("Alvenaria", "Elétrica", "Hidráulica",
              "Pintura", "Ferramentas")
estoque = []

def adicionar_produto(id_produto, nome, categoria, quantidade, preco):
    if preco ⩽0 or quantidade <0:
        print("Erro! Preço deve ser maior que 0 ...")
        return False

    nome_arrumado = nome.strip().title()
    produto = {
        "id": id_produto,
        "nome": nome_arrumado,
        "categoria": categoria,
        "quantidade": int(quantidade),
        "preco": float(preco),
    }
    estoque.append(produto)
    return True
```

TUPLA

CATEGORIAS não muda durante a execução e alimenta a caixa de escolha da tela.

LISTA + DICIONÁRIO

Cada material é um dicionário; todos ficam na lista estoque.

IF + STRING

Recusa preço zero ou negativo; strip().title() padroniza o nome.

CÓDIGO 2 · sistema.py · RF03 e RF04

Entrada, saída e saldo protegido

```
def controle_estoque(id_produto, quantidade_movimentada):
    for produto in estoque:
        if produto['id'] == str(id_produto):
            nova_quantidade = produto['quantidade'] + quantidade_movimentada
            if nova_quantidade < 0:
                print("Erro! Quantidade insuficiente no estoque")
                return False
            else:
                produto['quantidade'] = nova_quantidade
                salvar_dados()
                print(f"Estoque do  {produto['nome']} atualizado: ...")
                return True
    print(f"\nERRO: ID não encontrado, tente novamente\n")
    return False
```

FOR

Percorre o estoque até achar o ID pedido.

IF / ELSE · RF04

Se o novo saldo ficaria negativo, recusa e não altera nada.

RETORNO

Movimento aprovado grava o arquivo e devolve True; falha devolve False.

CÓDIGO 3 · sistema.py · RF07

Salvar e carregar em arquivo

```python
def salvar_dados():
    with open(ARQUIVO_DADOS, "w", encoding="utf-8") as database:
        for produto in estoque:
            linha = f"{produto['id']};{produto['nome']};..."
            database.write(linha)

def ler_arquivo():
    if not os.path.exists(ARQUIVO_DADOS):
        return
    with open(ARQUIVO_DADOS, "r", encoding="utf-8") as database:
        for linha in database:
            linha_limpa = linha.strip()
            if linha_limpa:
                partes_do_produto = linha_limpa.split(";")
                produto = {...}   # id, nome, categoria, qtd, preço
                estoque.append(produto)
```

ARQUIVO · ESCRITA

Modo “w” com UTF-8 para os acentos; um material por linha. O with fecha o arquivo sozinho.

ARQUIVO · LEITURA

Se o arquivo não existe, o sistema começa vazio e o cria ao salvar.

STRINGS

strip() limpa a linha e split(“;”) separa os campos; depois int() e float() convertem.

CÓDIGO 4 · interface.py · RF03 e RF06

Da tela para as regras

```
def movimentar(tipo):
    id_digitado = texto_mov_id.get().strip()
    quantidade_digitada = texto_mov_qtd.get().strip()
    ...  # valida campos vazios, inteiro e quantidade > 0

    fator = 1 if tipo == "entrada" else -1
    quantidade_final = quantidade_movimentada * fator

    sucesso = controle_estoque(id_digitado, quantidade_final)
    if sucesso:
        atualizar_tabela()
        texto_mov_id.delete(0, "end")
        texto_mov_qtd.delete(0, "end")
        mostrar_notificacao_mov("Sucesso! ...", cor="green")
    else:
        mostrar_notificacao_mov("Falha: ...", cor="red")
```

CALLBACK

Os botões chamam dar_entrada() e dar_saida(), que usam a mesma função movimentar().

SINAL

Saída vira fator -1; assim controle_estoque() trata entrada e saída do mesmo jeito.

FEEDBACK

Se deu certo, atualiza a tabela e limpa os campos; se não, mostra o erro em vermelho.

FECHAMENTO

Resultado e próximos passos

O MVP entrega

- 7 requisitos funcionais atendidos
- 4 histórias de usuário com confirmação
- Cadastro, entrada e saída com saldo protegido
- Busca, filtro e dados salvos em arquivo

Próximos passos

- Alerta de estoque mínimo
- Resumo do valor total em estoque
- Fazer uma versão Balconista e uma versão Gerente
- Tratar arquivo corrompido com try/except
- Histórico de movimentações

Obrigado! Perguntas?